

0.34. Un carré autour d'une lettre, avec carre

Une ligne, une lettre : `carre(10, 8, ALPHABET[0], 1, 1, 250, 1)` fait faire au A un tour en carré autour de la case (10, 8).

1. Ce qu'il faut comprendre

Une seule lettre, une seule ligne. Le A fait **un tour complet** autour de la case (10, 8), en carré, puis revient à sa place.

Ce qui est nouveau ici : `carre(x, y, tuile, taille, sens, vitesse, tours)`. Elle a sept réglages, mais pour commencer, ne regarde que les **trois premiers** :

- `10, 8` (`x`, `y`) : la case **autour de laquelle** on tourne, le **centre** du carré. La lettre part de là, et y revient.
- `ALPHABET[0]` (`tuile`) : ce qui tourne, la lettre A.

Les quatre autres restent simples ici : `taille 1`, `sens 1`, `vitesse 250`, `tours 1`. Ils veulent dire : le plus petit carré, dans le sens des aiguilles d'une montre, 4 pas par seconde, un seul tour. Les cours suivants les changent **un par un**.

Le trajet : le A part du centre (10, 8), fait un pas en diagonale jusqu'au coin en haut à gauche (9, 7), fait le tour du carré de **3 × 3** cases (droite, bas, gauche, haut), puis revient en diagonale au centre.

`carre` **ne rend rien** : la lettre finit exactement là où elle a commencé. Il n'y a ni `x =` ni variable à tenir.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <carre>` → [0.33.3. La fonction carre\(\) — tourner en carré](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1349 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
carre()         440 o ← main() — une fonction fournie par le langage
main()         38 o ← démarrage — ta fonction
VBlank          8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage   28 o ← image() — attend l'image suivante
AttendreVBlank  16 o ← démarrage — attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() — efface des cases
CopierTuiles    18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage — vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles — la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`,

DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()   × 4    52 o sur place, dans carre() + AdresseCase (10 o)
effacer() × 3    45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image()   × 4    12 o sur place, dans carre(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
carre()   × 1    31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// main : le programme commence ici. Les lignes entre { et } s'exécutent
// dans l'ordre, de haut en bas.
```

```
#include <carre>      // une tuile qui tourne en carré
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

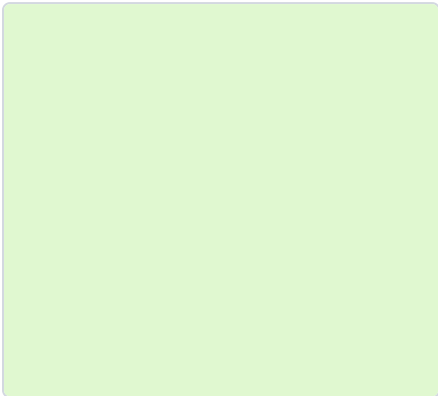
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Une seule ligne fait tout le carré. Ses sept réglages, dans l'ordre :
    //
    //  carre(10, 8, ALPHABET[0], 1, 1, 250, 1);
    //      | | |           | | | |
    //      | | |           | | |   +-- tours   : 1, un seul tour
    //      | | |           | | +----- vitesse : 250 millisecondes par pas,
    //      | | |           | |             soit 4 pas par seconde
    //      | | |           | +----- sens     : 1, comme les aiguilles
    //      | | |           |             d'une montre
    //      | | |           +----- taille    : 1, le plus petit carré (3 × 3)
    //      | | +----- tuile    : ce qui tourne ; ALPHABET[0]
    //      | |             est la 1re lettre, A
    //      | +----- y         : la LIGNE du centre, 8
    //      +----- x         : la COLONNE du centre, 10
    //
    // Pour ce premier cours, regarde surtout x, y et la tuile. Les quatre
    // autres réglages seront changés un par un dans les cours suivants.
    //
    // Ce que fait le A, un pas tous les quarts de seconde :
    //
    //      colonne :   9   10   11
    //      ligne 7 :   2 → 3 → 4
    //              ↑       ↓
    //      ligne 8 :   9   A   5
    //              ↑       ↓
    //      ligne 9 :   8 ← 7 ← 6
    //
    //  1.   le A apparaît au centre (10, 8)
    //  2.   un pas en diagonale : le coin en haut à gauche (9, 7)
    //  3 à 9. le tour, une case à la fois : droite, bas, gauche, haut...
    // 10.   ...jusqu'au coin (9, 7), d'où il était parti
    // 11.   un pas en diagonale : retour au centre (10, 8)
    carre(10, 8, ALPHABET[0], 1, 1, 250, 1);

    // Ici, carre a fini : le A est revenu au centre (10, 8), et il y reste.
    // while (true) tourne sans fin, pour que le programme ne s'arrête jamais.
    while (true) {
        image();           // attend l'image suivante (60 par seconde)
    }
}
```

3. Ce qui se passe, étape par étape

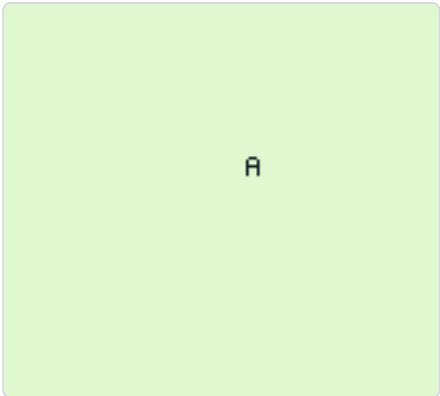
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui fait un petit tour en carré autour de la case (10, 8), puis s'arrête au centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les quatre coins, dans le sens des aiguilles d'une montre (9,7 11,7 11,9 9,9)
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.34.1. Un carré autour d'une lettre, avec carre — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++